Introduction on UML for Industrial Systems

Ansgar Radermacher / Asma Smaoui
ansgar.radermacher@cea.fr

Acknowledgments – contains material from my CEA colleagues
Shuai Li, Jérémie Tatibouët, François Terrier, Sébastien Gérard

Code generation from models (Model to Text)

1. **Code generation from models (Model to Text)**
2. **Role of static profiles**
3. **Language xtend**
4. **Apply to simplified Python/Keras generator**
5. **Test/debug an Eclipse plugin**
6. **Revision / questions**

Code generation from Models

- Motivation
  - Model = *executable* specification (not only documentation)
  - Provide (limited) early feedback
  - Map model elements to code

- Variants
  - Generate skeleton vs. generate full code
  - Synchronize model from code (roundtrip) vs. Code generation only

Recap: M2M and M2T transformations

- M2M = model to model transformation
  - Transform a source model into a target model

- Examples

  UML into another UML model (refinement)

  UML to an Ecore model

  …

Model to text (M2T) transformation

- Produce textual artefacts from a model, typically
  - Code
  - Formal verification languages
  - Documentation
  - Textual description languages (such as IDL)

- Here: only focus on code generation

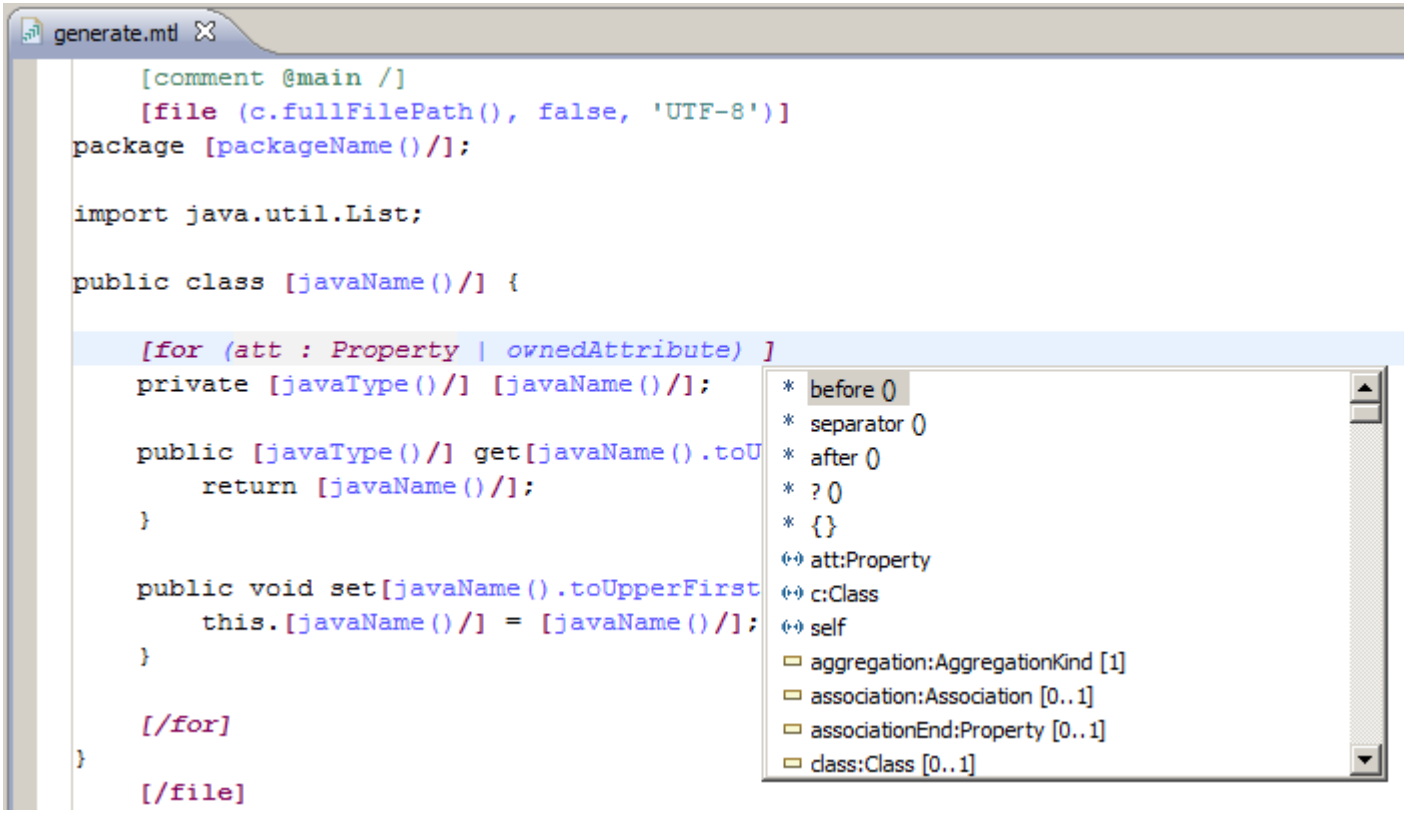
Implementation approaches

- Use QVT or ATL
- Papyrus SW designer
  - Combination of M2M and M2T transformations
    (quite simple code generators, advanced features via M2M)
- OMG M2T (https://www.omg.org/spec/MOFM2T)
- Specific languages – Acceleo and Xtend (more on Xtend later)

Acceleo

- “Generate anything from any EMF model”
- R&D result / support from OBEO

- Pragmatic
- Support for text templates
- Editor with assistance
- Interpreted language



Code generation from models (Model to Text)

1. **Code generation from models (Model to Text)**
2. **Role of static profiles**
3. **Language xtend**
4. **Apply to simplified Python/Keras generator**
5. **Test/debug an Eclipse plugin**
6. **Revision / questions**

Static profiles - Introduction

- We defined a profile, now need programmatic access to its **attributes**

- “Normal” profile – access attributes via generic methods; provided by common superclass `Element`

  - `getAppliedStereotype(“stereotype-name”) : Stereotype`
  - `getValue(stereotype, “attribute-name”) : Object`

Standard profiles – Attribute access




```
Stereotype conv1D =
      l1.getAppliedStereotype(”SimpleNN::Conv1D”);
int filters = (int) l1.getValue(conv1D, ”filters”);
```

Quiz



Disadvantage of generic stereotype access methods?



Stereotype or attribute name misspelled/renamed
⇨ code still compiles, runtime error



Generic method returns `Object`
⇨ need cast, no type safety!

Static profiles - Introduction

- We defined a profile, now need programmatic access to its **attributes**
- “Normal” profile – access attributes via generic methods
  - umlElement.getStereotype(“stereotype-name”)
  - umlElement.getValue(stereotype, “attribute-name”)
- Stereotype instance = Instance of a Java class?
- Ecore provides possibility to generate Java code from a (meta-) model

- Steps : UML => ecore model => code

Generated code …

Generated interface reflects inheritance hierarchy

```
public interface Conv1D extends Layer {
    /**
     * Returns the value of the '<em><b>Filters</b></em>' attribute.
     * …
     * @generated
     */
    int getFilters();

    /**
     * Sets value of '{@link …SimpleNN.Conv1D#getFilters <em>Filters</em>}' attribute.
     * …
     * @generated
     */
    void setFilters(int value);
}
```

Getter/setter for attributes

Static profiles – Attribute access




```
Stereotype conv1D =
      l1.getAppliedStereotype(”SimpleNN::Conv1D”);
int filters = (int) l1.getValue(conv1D, ”filters”);
```

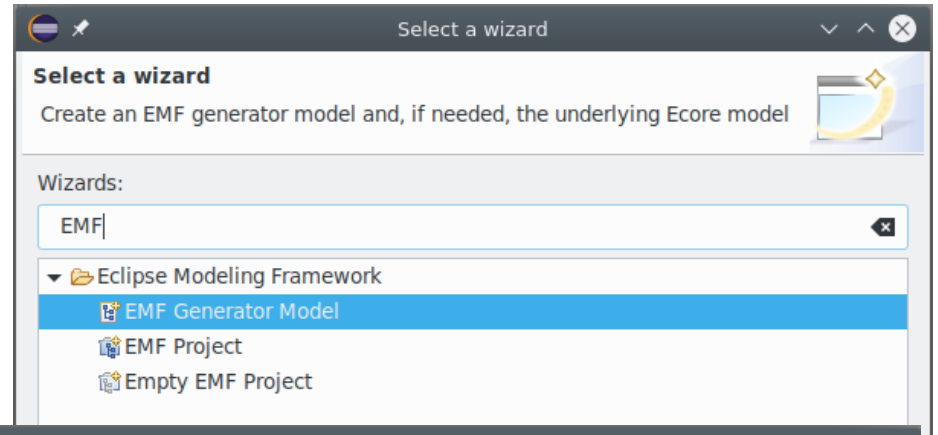


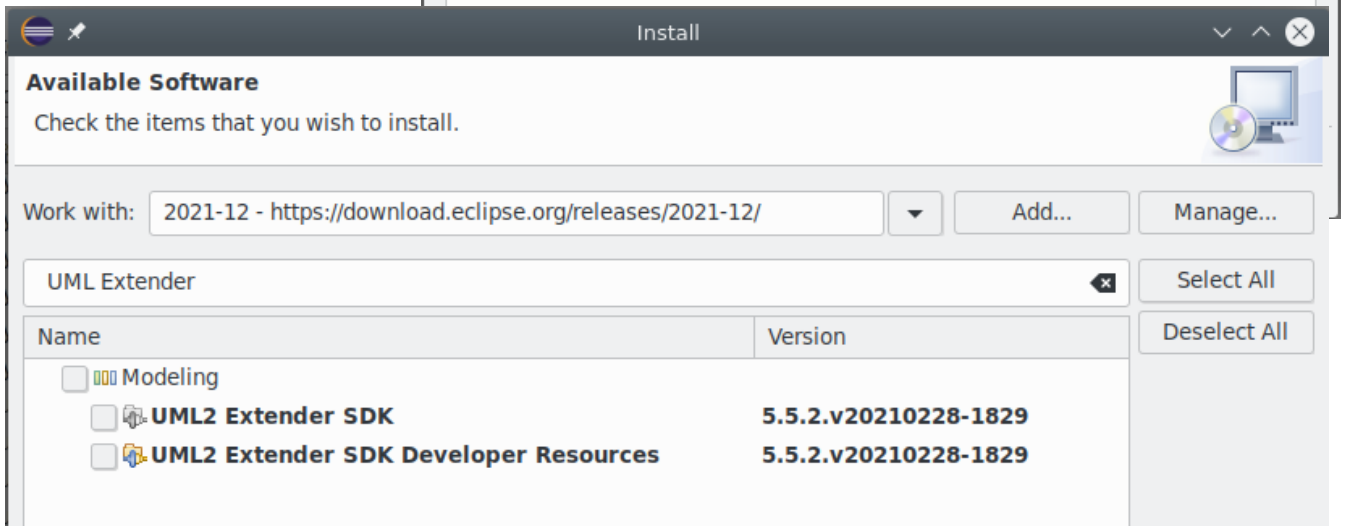
```
Conv1D conv1D =
      UMLUtil.getStereotypeApplication(l1, Conv1D.class);
int filters = conv1D.getFilters();
```

Type safe – return specific Conv1D interface

Exercise – generate a static profile

- Download plugin `org.eclipse.papyrus.simplenn.profile` from git
- Import the plugin into Eclipse workspace
- In sub-folder `profile`
  - Create new EMF Generator model
  - Use name SimpleNN.profile.genmodel
  - Import option: UML model
  - Load model SimpleNN.profile.uml
  - Precondition install UML2 extender SDK





Code generation from models (Model to Text)

1. **Code generation from models (Model to Text)**
2. **Role of static profiles**
3. **Language xtend**
4. **Apply to simplified Python/Keras generator**
5. **Test/debug an Eclipse plugin**
6. **Revision / questions**

Xtend language

- Java dialect, compiles into (readable) Java 8 source code.
- Use existing Java libraries seamlessly.
- Macros, lambdas, operator overloading and many more modern language features.
- Type inference – no need to type in most case
- Text Templates (that´s why we´re looking at it)

Type inference & syntactic sugar

- Replace
  `Type a = ...` with
  `var a = ...` or `val a =`

- The type with we infered from the expression on the right.
- Access “get” method as if an attribute – useful for generated MM methods
- Static extension, replace
  `UtilClass.fctA(a)` with
  `a.fctA`
- Clever indentation

Xtend – text templates

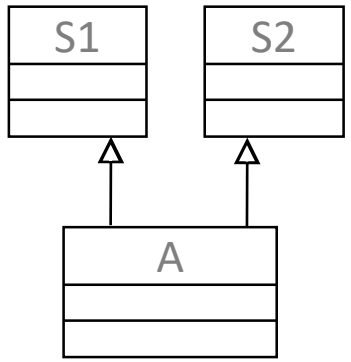
- Example
- '''class «c.name» extends
  «FOR superclass : c. superclasses SEPARATOR “, “»
  «superclass.name»
  «ENDFOR»'''‘

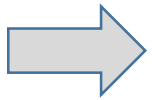
Shorthand for method getName()

Shorthand for method getSuperClasses()

FOR loop

Separator character



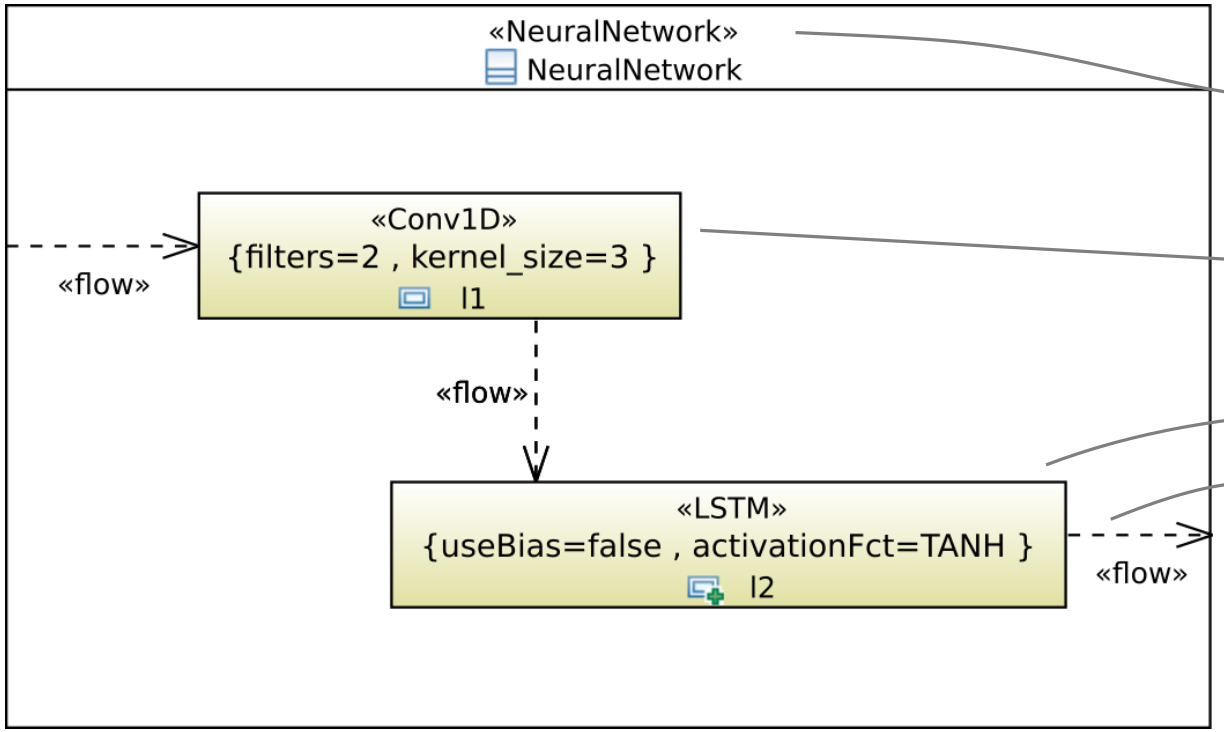


class A extends
S1, S2

Code generation from models (Model to Text)

1. **Code generation from models (Model to Text)**
2. **Role of static profiles**
3. **Language xtend**
4. **Apply to simplified Python/Keras generator**
5. **Test/debug an Eclipse plugin**
6. **Revision / questions**
7. **Homework**

Application for Neural Network




```python
import tensorflow as tf
from tensorflow import keras

class NeuralNetwork(tf.keras.Model):
    def __init_(self):
        super(module.base_Class.name», self).__init__();

        self.l1 = tf.keras.layers.Conv1D(2, 3);
        self.l2 = tf.keras.layers.LSTM(false, TANH);

    def call(self, inputs):
        x0 = self.l1(input)
        output = self.l2(x0)
        return output;

testModule = NeuralNetwork()
testModule.compile(name="NeuralNetwork")
```

input model (from cours2)
Stereotyped class with parts
composite structure diagram

wanted output
Python/Keras code

Code generator, part 1 (xtend)

```
/**
 * Generate a model with different elements necessary for the compilation
 */
static def genModule(NeuralNetwork module) '''
   import tensorflow as tf
   from tensorflow import keras

   class «module.base_Class.name»(tf.keras.Model):
      def __init_(self):
         super(module.base_Class.name», self).__init__();

         «FOR attribute : module.base_Class.attributes»
               «val layer = UMLUtil.getStereotypeApplication(attribute, Layer)»
               self.«attribute.name» = «layer.genLayer»
         «ENDFOR»
```

Text template

Access to (meta-) model

Control loop (over attributes)

Code generator, part 2 (xtend)

```
…
    def call(self, inputs):
        «var i = 0»
        «val flowList = module.base_Class.getFlows()»
        «FOR layer : flowList»
            «val previous = (i == 0) ? "input" : '''x«i-1»'''»
            «val current = (i == flowList.size-1) ? "output" : '''x«i++»'''»
            «current» = self.«layer.name»(«previous»)
        «ENDFOR»
        return output;

testModule = NeuralNetwork()
testModule.compile(name="NeuralNetwork")
```

Auxiliary function

input, intermediate, output

Code generator, part 3 (xtend)

```
…
// Generate each layer according to the model
static def genLayer(Layer layer) {
   if (layer instanceof LSTM) {
      genLstm(layer as LSTM)
   } else if (layer instanceof Conv1D) {
      genConvolutionLayer(layer as Conv1D)
   }
}

// Generate convolution layer according to the model
static def genConvolutionLayer(Conv1D conv1d) '''
   tf.keras.layers.Conv1D(«conv1d.filters», «conv1d.kernel_size»);
'''
```

Use static profile

Access attributes of static profile

Exercise

- Copy plugin `org.eclipse.papyrus.simplenn.gen.keras` into your workspace (and import it) – it´s on the git (tutos folder)
- Some information about Eclipse plugins (a bit off-topic)
  - OSGI MANIFEST.MF – plugin properties, dependencies
  - plugin.xml - Extension points & properties (for menus, etc.)

Acceleo vs. Xtend

- Text templates are relatively similar in both languages

- Advantages Acceleo
  - Dedicated to M2T, output file creation support
- Advantages Xtend
  - Compiled into Java => 1. Fast
    2. Use any Java function w/o specific declarations,
    3. Easy to debug with standard tooling
  - Standard language for code generators for Papyrus

What is round-trip engineering?



"The ability to automatically maintain the consistency of multiple, changing software artifacts, in software development environments/tools, is commonly referred to as round-trip engineering"

- Related to traditional two software engineering disciplines:
  - Forward engineering: creating software from specifications
  - Reverse engineering: creating specifications from existing software
- Round-trip engineering adds synchronization of existing artifacts that evolved concurrently by incrementally updating each artifact to propagate changes made to the other artifact
- Round-trip generalizes both forward and reverse engineering

[Sendall04] S. Sendall and J. Küster. Taming model round-trip engineering. In *Proceedings of Workshop on Best Practices for Model-Driven Software Development*, Vancouver, Canada, 2004.

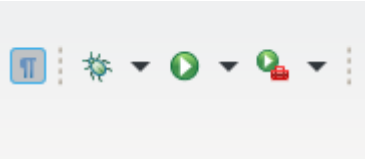
Code generation from models (Model to Text)

1. **Code generation from models (Model to Text)**
2. **Role of static profiles**
3. **Language xtend**
4. **Apply to simplified Python/Keras generator**
5. **Test/debug an Eclipse plugin**
6. **Revision / questions**

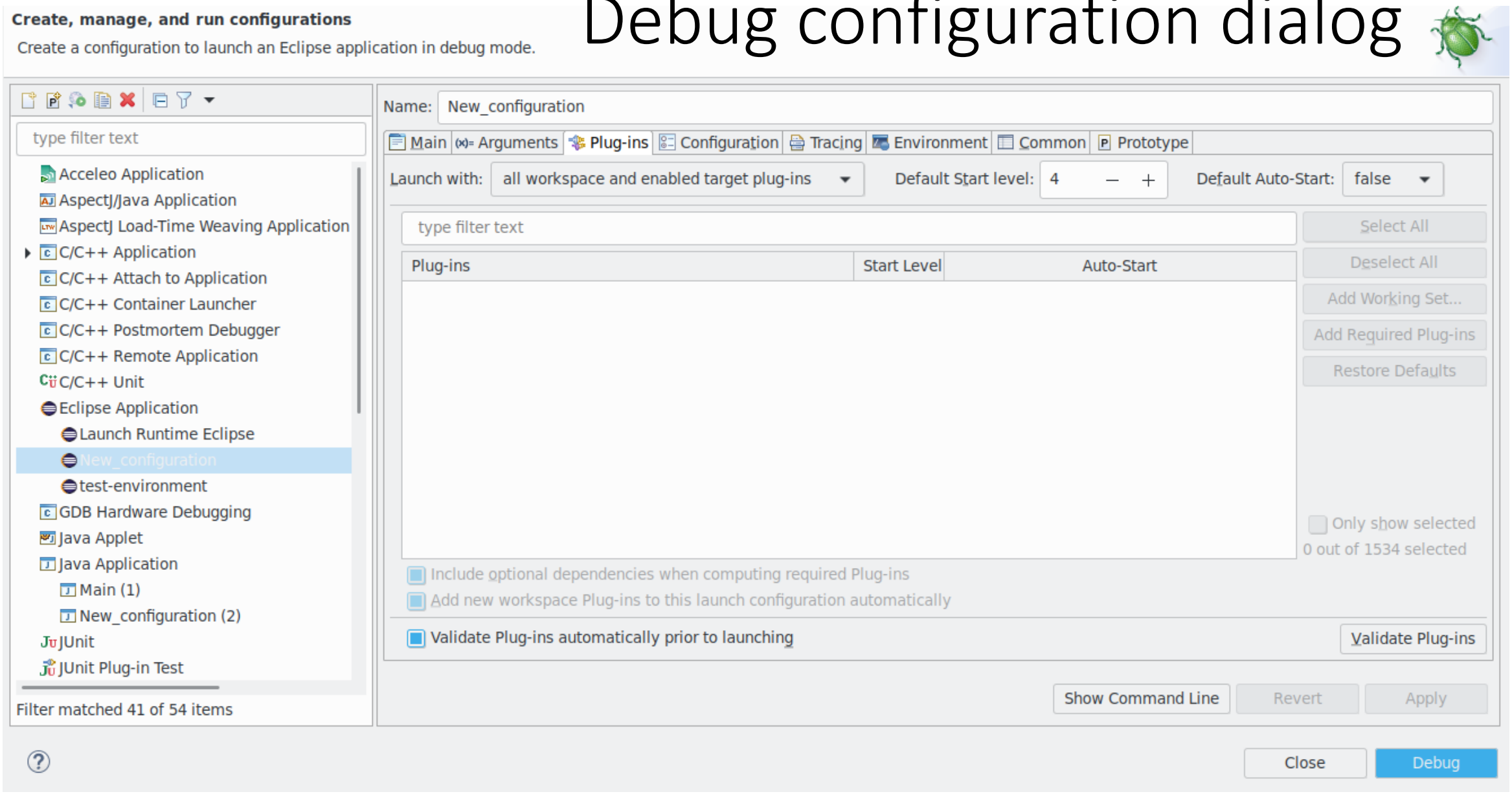
Debugging an Eclipse (workspace) plugin

- Eclipse only takes installed plugins into account
- But a ***new/2nd Eclipse*** instance will contain your workspace plugins
- Behaves like a normal Eclipse, runs in its own workspace

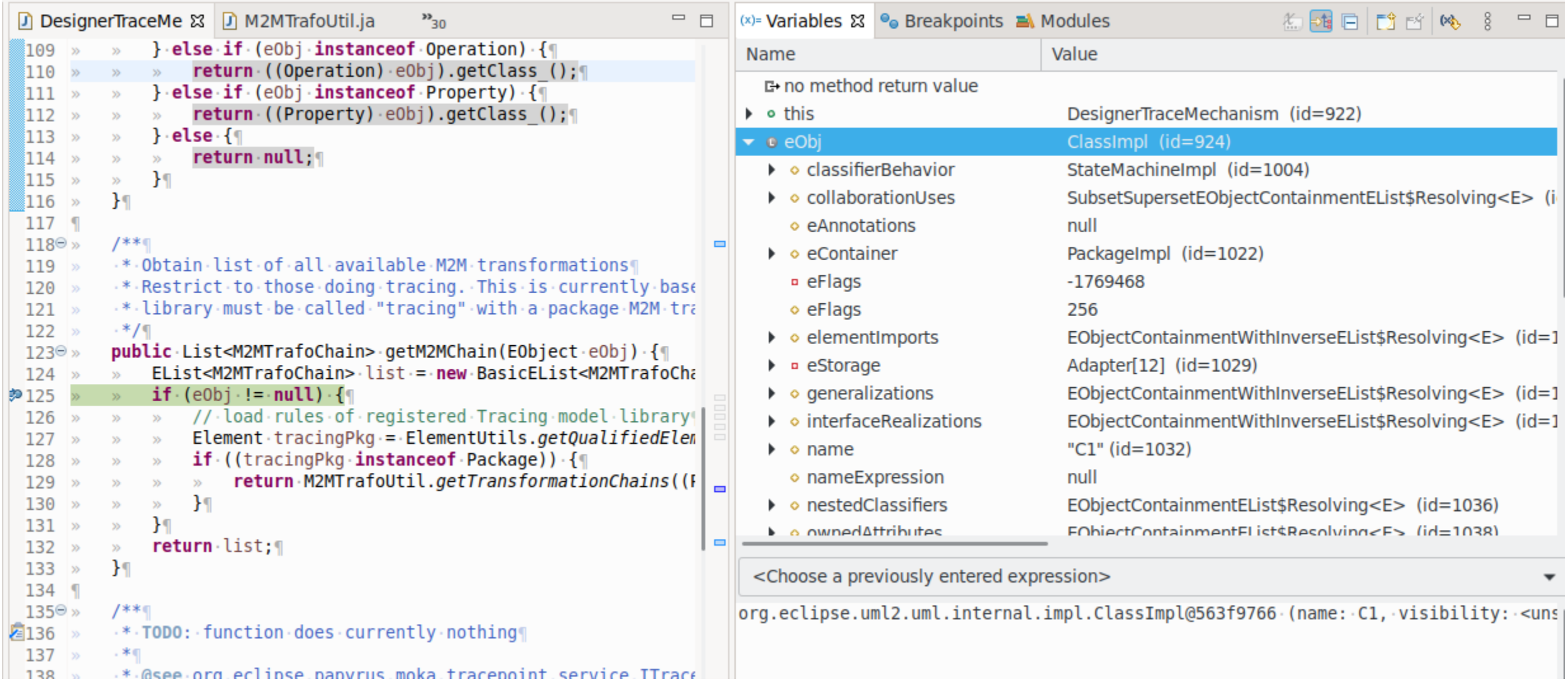
- Create a new Eclipse instance and run it
- Open the Debug Configurations dialog:
  - Menu **Run -> Debug configurations**
  - Via **Tool bar**



Debug configuration dialog

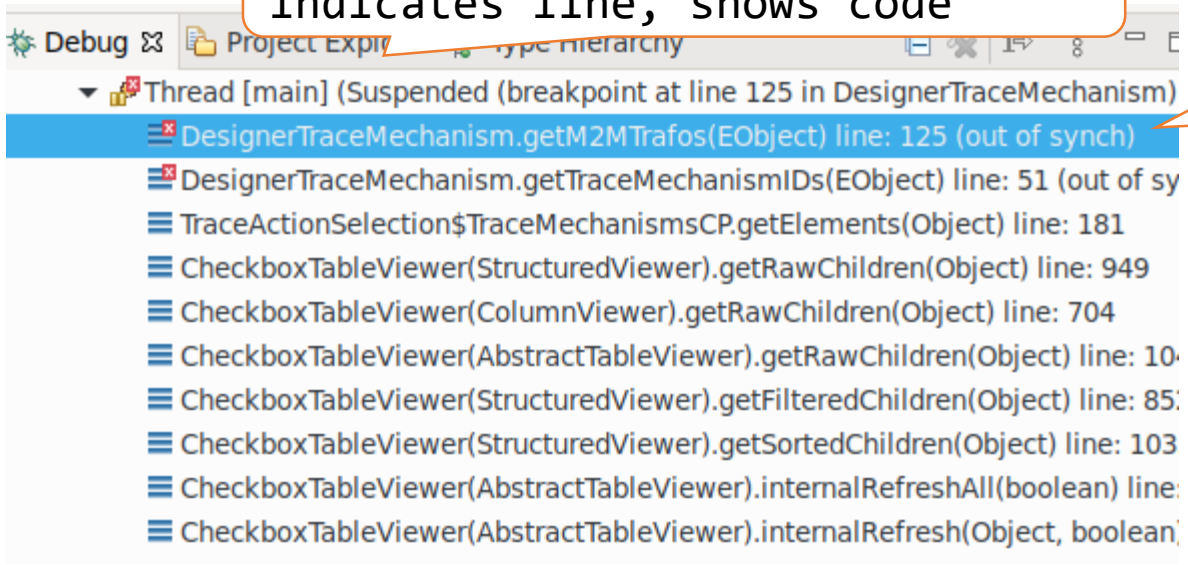


Place a breakpoint, inspect variables



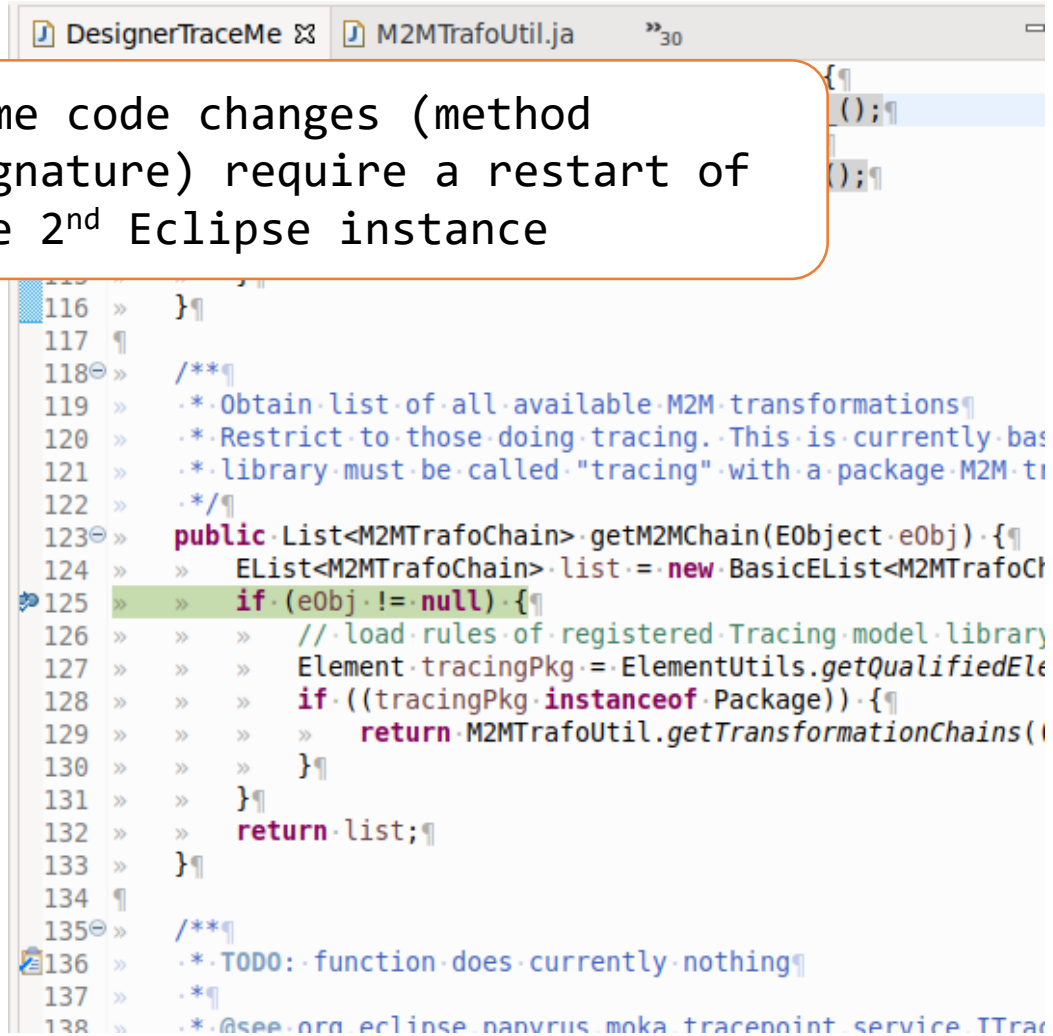
Stack trace

Debugger stopped on breakpoint, indicates line, shows code



User can navigate to any element in the hierarchy

Some code changes (method signature) require a restart of the 2nd Eclipse instance



Code generation from models (Model to Text)

1. **Code generation from models (Model to Text)**
2. **Role of static profiles**
3. **Language xtend**
4. **Apply to simplified Python/Keras generator**
5. **Test/debug an Eclipse plugin**
6. **Revision / questions**

Revision – What we´ve learned …

- Motivation – why modeling?
- Domain specific vs. general purpose languages
- Textual vs. graphical modeling languages
- Meta-models
- UML diagrams
  - Global functions – Use case diagram
  - Interactions modeling – sequence diagrams
  - Different modes/states – state machine diagram
  - Component-based modeling – composite structure diagram
  - Behavior – Activity diagrams

Revision – We´ve learned …

- Domain specific (modeling) languages – DSLs / DSMLs
  - Editor generation with xtext
- Domain specific modeling with UML
  - Real-time and embedded systems – MARTE profile
  - Extend and Restrict the language – UML profiles with OCL rules
  - Exploit the language for code generation – static profiles
- Model transformations and code generation
  - OMG QVT and ATL
  - Code generation (running/debugging plugins)
- Practical use of UML in Eclipse/Papyrus, debugging

Papyrus (for Robotics)

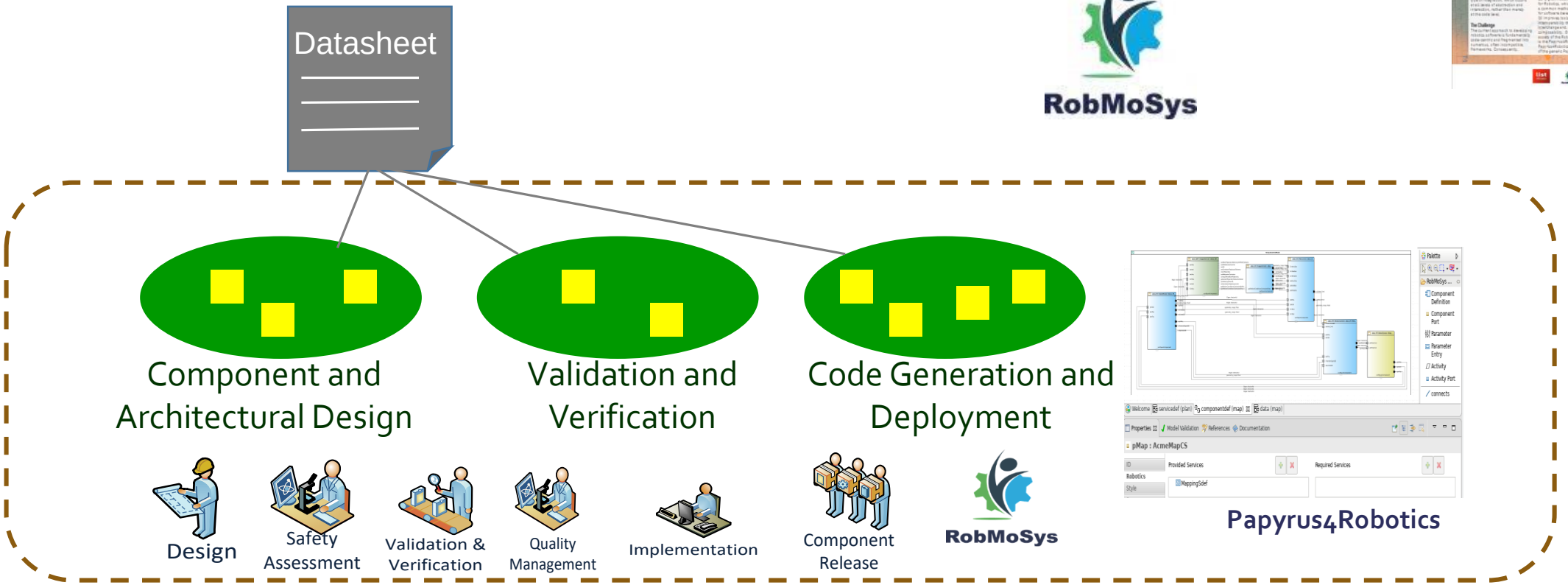
Papyrus for Robotics – customization for the robotics domain

https://eclipse.org/papyrus/components/robotics

Youtube channel: https://www.youtube.com/c/PapyrusEclipseUML

RobMoSys aligned – European project https://robmosys.eu/





The Robot Operating System (ROS)

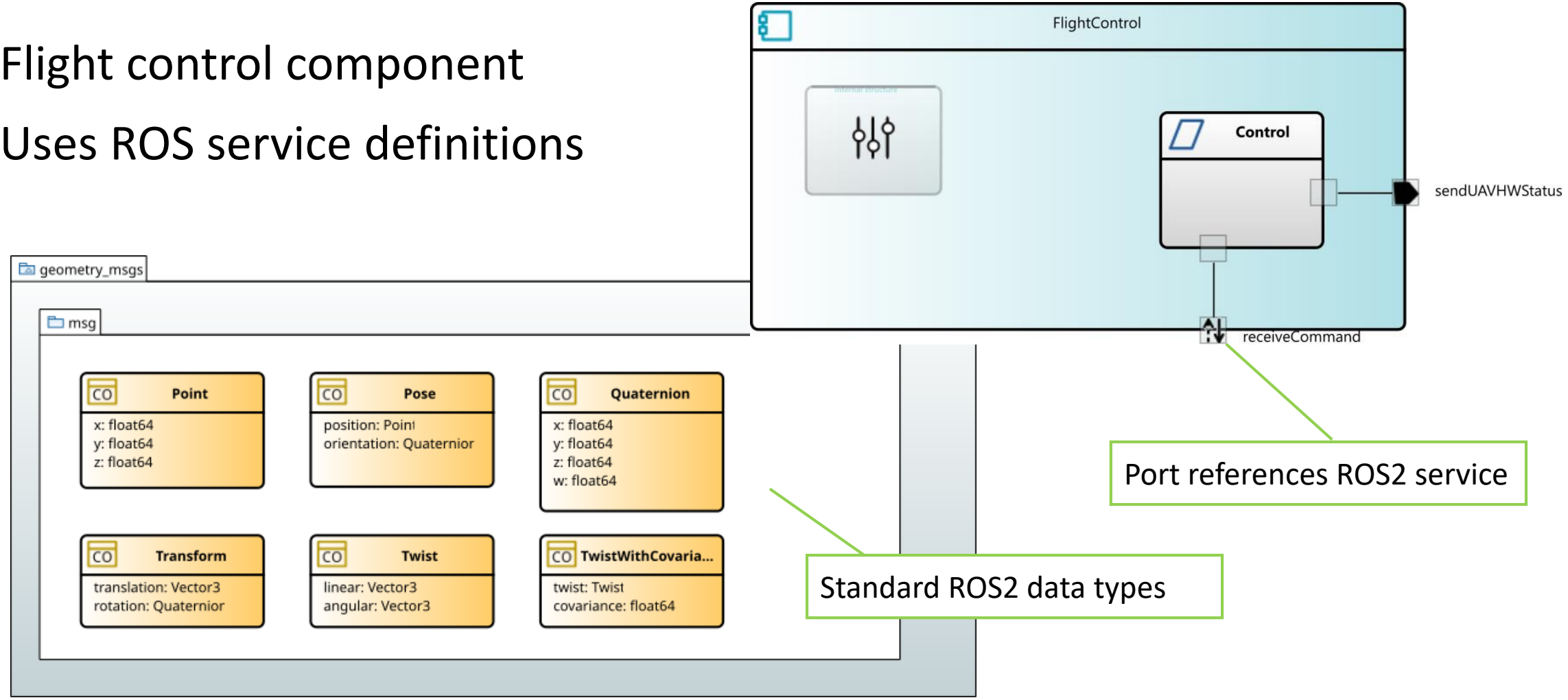
- https://index.ros.org/doc/ros2/
- Set of software libraries and tools for building robot applications.
- Wide range (drivers, algorithms, visualization tools …), open source.
- ROS 1 was started in 2007
- ROS 2 – reduced footprint, based on DDS middleware, better real-time support
- microROS for resource-constrained systems

⇨ Growing number of companies migrating to ROS2
⇨ Papyrus for Robotics supports code generation for ROS2

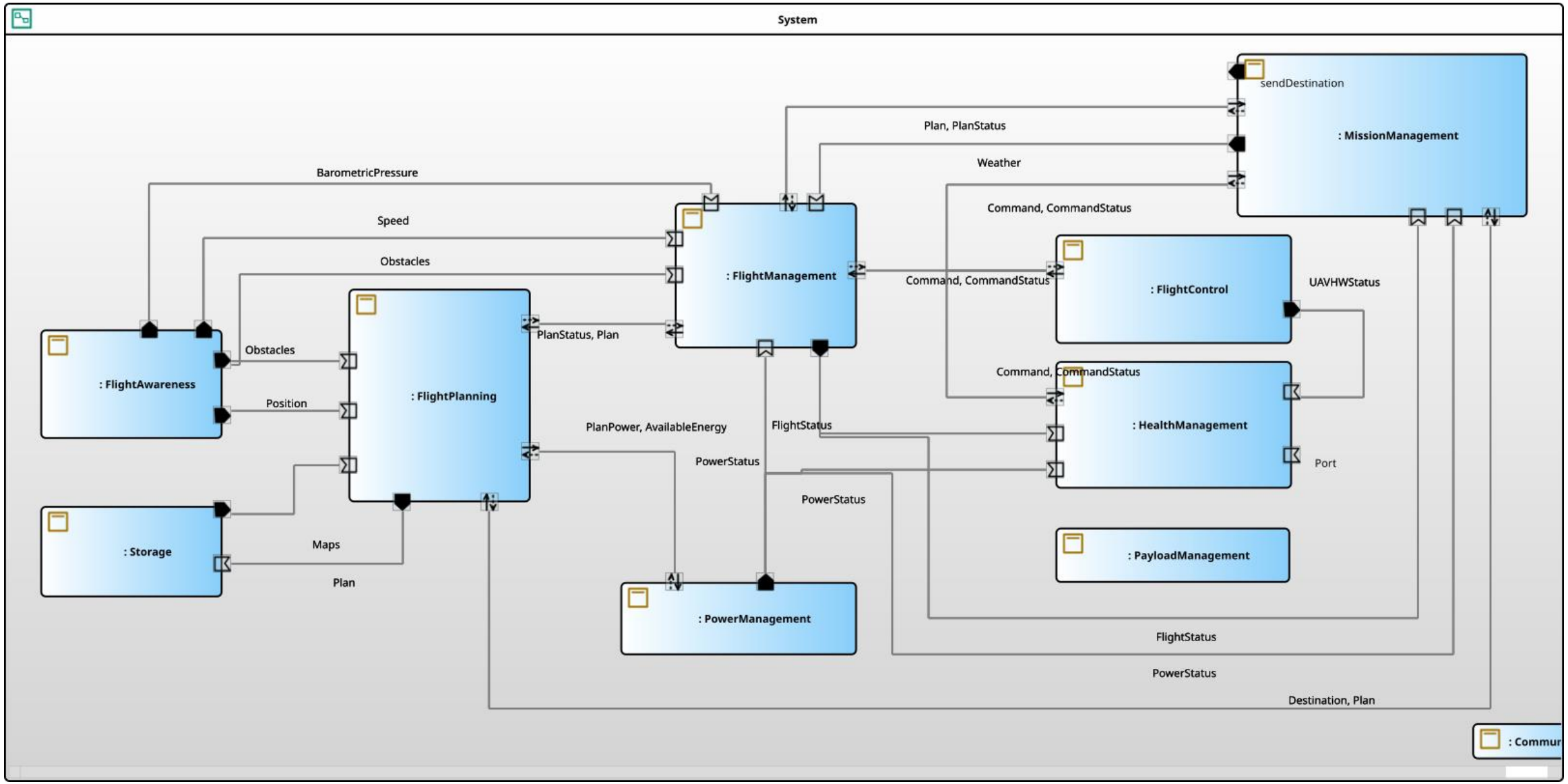
ROS-MDD ROSin

Examples in Papyrus for Robotics

- Flight control component
- Uses ROS service definitions



Example System model



Feedback

- Which topic did you consider most important? (most interesting?)
- What did you understand / what was too difficult to understand?
- Too many topics, just right or something important was missing?
- Quality of course & material?
- Importance of modeling skills for getting hired?